UNIVERSITÄT POTSDAM

Course Materials · First Edition




# Railway Routing & Scheduling

*A Learning Resource for Using Answer Set Programming in Flatland*

R. Murphy

*Course Materials*

Compiled: October 19, 2026

# Contents

**Introduction** **5**

**1 History of railway routing and scheduling** **7**
1.1 Classical literature . . . 7
1.2 Modern approaches . . . 7

**2 Related fields** **9**
2.1 Multi-agent pathfinding . . . 9
2.2 Job shop scheduling . . . 10

**3 Flatland challenge** **11**

**4 Answer Set Programming (ASP)** **13**

**5 A formal definition of Flatland** **15**
5.1 Crosscutting concepts . . . 15
5.2 Grid environment . . . 15
5.3 Trains . . . 17
5.4 Actions . . . 17
5.5 Graph translations . . . 18

**6 Flaspland toolkit** **19**

**7 Baseline encodings** **21**
7.1 Auxiliary encodings . . . 21
7.1.1 directions.lp . . . 21
7.1.2 tracks.lp . . . 21
7.1.3 grid-functions.lp . . . 22
7.1.4 aux.lp . . . 22
7.2 Graph translations . . . 23
7.2.1 subnodes.lp . . . 23
7.2.2 edge-functions.lp . . . 23
7.2.3 hypergraph.lp . . . 23
7.3 Pathfinding encodings . . . 23

# Introduction

This textbook is a resource for students in the Railway Scheduling course at Universität Potsdam. The goal of the book is to introduce students to both the Flatland challenge and the corresponding Flaspland toolkit, developed here at the university to bridge the gap between the existing challenge code and Answer Set Programming using the clingo language. Students who attend the course are expected to have working knowledge of Answer Set Programming; therefore, an understanding of concepts including clingo syntax, choice rules, constraints, aggregates, and optimization is assumed. It is also beneficial to have a rudimentary understanding of the Python programming language. Students do not need to develop their own Python code, but since the original Flatland challenge was developed using Python, it may be advantageous to be able to interpret it. Existing knowledge of graph theory, scheduling, or multi-agent pathfinding is an added plus.

## How this book is organized

The book is organized in two broad sections. First, there is general background on the principles of railway routing and scheduling, described in the first three chapters:

Chapter 1: A historical and modern review on railway routing and scheduling as an academic discipline

Chapter 2: A glimpse into two similar problems, whose foundations may provide a basis for solving railway routing and scheduling

Chapter 3: An introduction to the Flatland challenge, as well as its core components

Second, there is the more technical side, described in the final four chapters:

Chapter 4: A brief explanation of Answer Set Programming

Chapter 5: An overview of a formal definition of Flatland, which is especially helpful for solving it using Answer Set Programming

Chapter 6: A guide to the Flaspland toolkit, which allows users to leverage clingo as a solution to Flatland problems

Chapter 7: A set of example encodings that help lay the groundwork for solving more complex tasks using clingo

## Software and resources

Please take advantage of the following resources:

- Course Moodle page: https://moodle2.uni-potsdam.de/course/view.php?id=37437
- Flaspland toolkit repository: https://github.com/potassco/flaspland-toolkit
- Flaspland example encodings: https://github.com/potassco/flaspland-encodings
- Potassco resources for learning Answer Set Programming: https://potassco.org/resources/
- Flatland challenge: https://flatland.aicrowd.com/intro.html

# Chapter 1

# History of railway routing and scheduling

The history of railway routing and scheduling from an academic perspective dates back to the middle of the twentieth century.

## 1.1 Classical literature

Research from the twentieth century focused primarily on individual aspects of railway routing and scheduling problems, whose approaches generally belonged to the field of operations research. In [9], dynamic programming is used to find a minimum cost schedule. In [4], integer programming is used to calculate heuristics for quickly finding feasible solutions for scheduling trains. In [3], Lagrangian relaxation is used to solve the railway timetabling problem.

Although published in 2011, **Scheduling and Rescheduling of Railway Operations: A Review and Expository Analysis** [8] cites dozens of papers from the 1980s and 1990s and provides an overview of terminology, a hierarchical organization system (see Figure 1.1), and a classification structure for sub-problems within railway operations (see Figure 1.2).

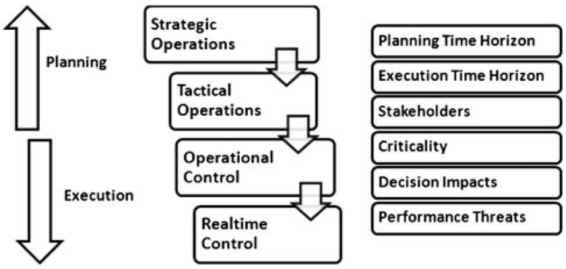



Figure 1.1: Hierarchy of railway operations from planning to execution, and the dimensions along which they differ [8].

The research draws a line between planning activities and execution activities, which are (1) plans developed ahead of time under ideal conditions and (2) actions with the purpose of carrying out the ideal plans or actions to alleviate deviations from the ideal plans, respectively. This important distinction is demonstrated by two similar but distinct problems: *scheduling* and *rescheduling*. These two problems provide the foundation of the Flatland challenge, which will be discussed in greater detail in Chapter 3.

## 1.2 Modern approaches

Modern approaches from the twenty-first century have focused on more complex railway routing and scheduling problems, which are often solved using a combination of operations research and artificial intelligence techniques. Chief among this research are two papers we follow closely:




Figure 1.2: Classification of railway operations into strategic, tactical, operational, and real-time activities [8].

1. **Routing and Scheduling in Answer Set Programming** [2], which provides a series of approaches for solving routing and scheduling problems in the multi-agent pathfinding domain

2. **Train Scheduling with Hybrid Answer Set Programming** [1], which provides a real-world train scheduling application

The first paper presents a variety of answer set programming (ASP) approaches to routing and scheduling problems in the context of multi-agent pathfinding, a related domain discussed in Chapter 2. Although the underlying environments differ in structure, the principles of pathfinding can be applied to problems within this domain. Additionally, the paper proposes alternative methods for dealing with time, which stand to yield significant performance enhancements.

The second paper presents a real-world application of ASP approaches on the Swiss Federal Railway system. It combines routing, scheduling, optimization, heuristics, and preprocessing techniques to form a comprehensive system. The overall problem is solved as three distinct tasks: routing, conflict resolution, and scheduling. Both papers make use of difference logic, which supplements traditional ASP techniques.

# Chapter 2

# Related fields

From a theoretical and computational perspective, railway routing and scheduling problems share similarities with other well-studied problems in operations research and computer science. Routing and scheduling form two sides of the same coin, and two areas, namely multi-agent pathfinding and job shop scheduling, provide well-researched techniques for solving each of these two sides.

## 2.1 Multi-agent pathfinding

In multi-agent pathfinding (MAPF) problems, the task is to plan paths for multiple agents, such that each agent can follow its path without colliding with another agent [10]. Modern applications of MAPF include automated warehouses, autonomous vehicles, and robotics. Different sets of assumptions can be applied to MAPF scenarios, such as modifications to the underlying environment or changes to the objective of the agents. Many similarities exist between MAPF and railway routing and scheduling problems, such as the need to avoid collisions between agents (trains), the structure of the underlying environment (railway networks), and the adherence to a set of deadlines (train schedules). There are two primary differences between MAPF and railway routing and scheduling problems. First, in MAPF, agents are ordinarily granted more freedom of movement throughout an environment. Often, agents are free to move to an adjacent cell in any direction, provided the cell is unoccupied and the move does not result in a collision with another agent or obstacle. Trains, on the other hand, are constrained to move along tracks in a railway network. They also retain a specific orientation and cannot move backward. Second, MAPF environments are inherently restrictive, meaning an agent is free to safely move to any unoccupied adjacent cell *unless there is an obstacle preventing it from doing so.* Railway networks, on the other hand, are inherently permissive, meaning an agent can only move to an adjacent cell *if there is a track connecting the two cells.*

Railway routing can generally be viewed as a more constrained version of MAPF, where the underlying environment is a railway network and the agents are trains. Within a given horizon (i.e. a maximum number of time steps during a simulation), the set of solutions to a railway routing problem can be viewed as a subset of the set of solutions to a corresponding MAPF problem. In principle, this is advantageous in the sense that similar techniques can be applied to both problems, with the added benefit of railway routing problems having more constrained search spaces.

Railway routing also borrows two important concepts from MAPF: **vertex conflicts** and **edge conflicts**. To translate these ideas into the Flatland domain, we say a vertex conflict arises when two trains simultaneously occupy the same cell. We say an edge conflict, sometimes called a swap conflict, arises when two trains simultaneously swap cells; while they never occupy the same cell

together, they would simultaneously occupy the space between their cells. Such conflicts model the effects of collisions between two trains, which in real life cannot occupy the same physical space and cannot pass through each other.

## 2.2 Job shop scheduling

Job shop scheduling (JSS) is a core problem faced in manufacturing industries. It is defined by a set of jobs – each of which has a duration – and a limited number of resources [5]. The aim is to find a schedule that matches jobs to resources, subject to some set of constraints and possibly to some optimization criteria.

Railway scheduling overlaps in part with JSS, viewing trains as jobs and portions of the rail network as resources. In particular, trains may have specified departure and arrival times, out of which arise durations similar to those of jobs.

# Chapter 3

# Flatland challenge



Figure 3.1: A Flatland environment: a grid-based railway network of tracks, switches, and stations.

The Flatland challenge is a global railway-scheduling challenge founded as a collaboration between the Swiss Federal Railways (SBB) and AIcrowd. It was born out of a need for collaboratively sourcing technological solutions to routing and scheduling problems facing large railway operators. The first challenge took place in 2019, with improvements and changes made for the subsequent challenges in 2020 and 2021.

There are two key problems facing SBB and other large railway operators:

1. Vehicle Scheduling Problem (VSP)
2. Vehicle Rescheduling Problem (VRSP)

The research group at SBB developed an automated traffic management system known as TMS, which simulated the railway infrastructure and optimized the flow of traffic by selecting train routes and deciding how to order trains at switches. This is the VSP in action, which is a well-researched field, going beyond just railway networks. Although the TMS was successful for handling VSP tasks, railway operators often face situations that no longer match the ideal scenarios simulated by such tools. When breakdowns, delays, or other interruptions occur, trains deviate from their schedules, which could have downstream effects, including collisions, if not handled adequately. Solving problems under such circumstances is the VRSP in action.

The VRSP is computationally quite challenging to solve, so SBB developed Flatland as a simpler simulation tool, so that potential approaches could focus on only the most important aspects of the problem. The initial goal of Flatland was to foster innovation with respect to operations

research (OR) and reinforcement learning (RL) methods, as well as combinations thereof, to provide novel solutions for the VRSP on railway networks. Early versions of Flatland were tailored to reinforcement learning in particular, including provisions for reward functions, but non-RL methods proved more successful on the leaderboards.

The reason optimization approaches tend to outperform reinforcement learning approaches may have to do with deadlocks. A train experiencing a deadlock has no feasible path to its goal. Such a situation arises when at least two trains make navigational decisions that block each other's path. Typically, this happens when two trains travel toward each other down a single corridor with no alternative path. Since trains cannot move in reverse, there is no way to move forward without incurring a vertex or edge conflict. The trains subsequently block the section of track for the remainder of the simulation, potentially causing deadlocks for other trains. In theory this can happen during the initial planning phase, but approaches such as ASP can forbid this from taking place by implementing proper constraints, guaranteeing conflict-free and deadlock-free paths. Approaches in the Flatland challenge become more exposed during the replanning phases following malfunctions. Depending on how replanning is handled, trains that are still operating may proceed in a manner that violates those original deadlock-free guarantees. The winners of the 2020 Flatland challenge explain their suspicions about why optimization approaches have an advantage here. They claim that optimization approaches can reason globally about trains and about the broader concept of paths, whereas reinforcement learning approaches cannot [6]. Reinforcement learning approaches cannot predict future deadlocks without performing this direct form of reasoning, and individual trains may cause deadlocks without considering global consequences. As mentioned, if large segments are effectively rendered impassable for the entire simulation, further trains acting without a global perspective may find themselves deadlocked as well. It is difficult to consider all situations in which a deadlock would arise without including a global observation of all paths. As the number of trains grows, the number of situations leading to deadlocks increases non-linearly, giving an edge to optimization approaches in this regard.

# Chapter 4

# Answer Set Programming (ASP)

Answer Set Programming (ASP) is a declarative programming paradigm that is particularly well-suited for solving combinatorial problems, such as those encountered in railway routing and scheduling. In ASP, problems are encoded as logic programs, and solutions are represented as *answer sets* that satisfy given constraints.

Because ASP has been successfully applied to a wide range of combinatorial problems, including the similar problems already discussed in Chapter 2, it is a natural choice for addressing the challenges posed by railway routing and scheduling. This is heavily supported by the existing precedent for using ASP in the context of railway scheduling, as demonstrated by the work in [1].

Similar to ASP, approaches from the broader logic community, such as Constraint Programming (CP) and Integer Linear Programming (ILP), have also been applied to similar problems and may be of interest to those seeking to explore novel methods in ASP.

# Chapter 5

# A formal definition of Flatland

An extensive formalization has been undertaken in our published work [7]. A few selected elements are reproduced here informally for convenience.

## 5.1 Crosscutting concepts

- **Directions.** We define a set of cardinal directions $D = \{n, s, e, w\}$ to describe the orientation of trains within the Flatland environment.
- **Tracks.** We define a set $T$ of tracks, each of which can be derived from basic types that may be rotated in increments of 90°. These tracks are shown in Figure 5.1.
- **Transitions.** We define a relation $H_t$ for each track type $t \in T$ comprising a set of transitions of the form $(d, d')$, where $d, d' \in D$. Within each relation, each transition corresponds to a complementary transition.

## 5.2 Grid environment

- **Cells.** A railway network is an $n \times m$ grid of cells $C$. We associate each cell $c \in C$ with a coordinate position and a track type.
- **Adjacency.** A cell $c$ is adjacent to a cell $c'$ if they lie in the same row and are offset by one column, or if they lie in the same column and are offset by one row. We use the directions in $D$ to describe the relative positions of adjacent cells.
- **Connectedness.** We define that a cell $c$ is connected in the direction $d \in D$ to a cell $c'$, notated as $c \overset{d}{\hookrightarrow} c'$, if:
  1. $c$ is adjacent to $c'$ in the direction $d$
  2. $c$ has a transition $(d', d) \in H_{t_{(c)}}$
  3. $c'$ has a transition $(d, d'') \in H_{t_{(c')}}$

In light of these definitions, it is clear to see the importance of the relationship between cells and directions. To tie these concepts together, we define a **transit pair** as a tuple $(c, d)$, where $c \in C$ and $d \in D$. This helps define a **transit path** through the environment as a sequence of distinct transit pairs $((c_i, d_i))_{i=0}^{n}$ where $n > 0$ such that for all $0 \leq i < n$:

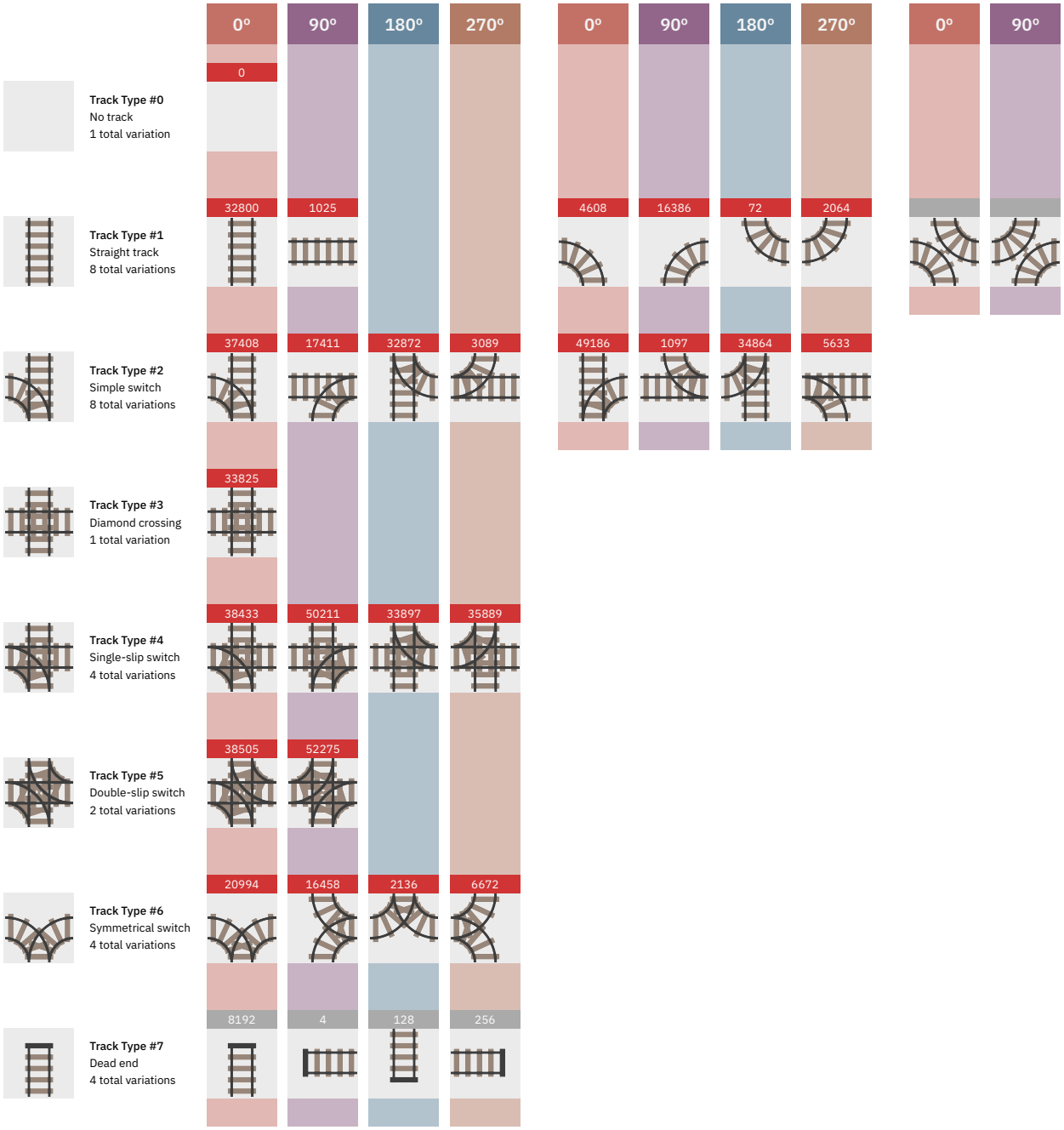



Figure 5.1: Flatland track types and their rotations, each labelled with the cell value Flatland uses to encode it.

1. $c_i \overset{d_i}{\hookrightarrow} c_{i+1}$
2. $(d_i, d_{i+1}) \in H_{t_{(c_{i+1})}}$

## 5.3 Trains

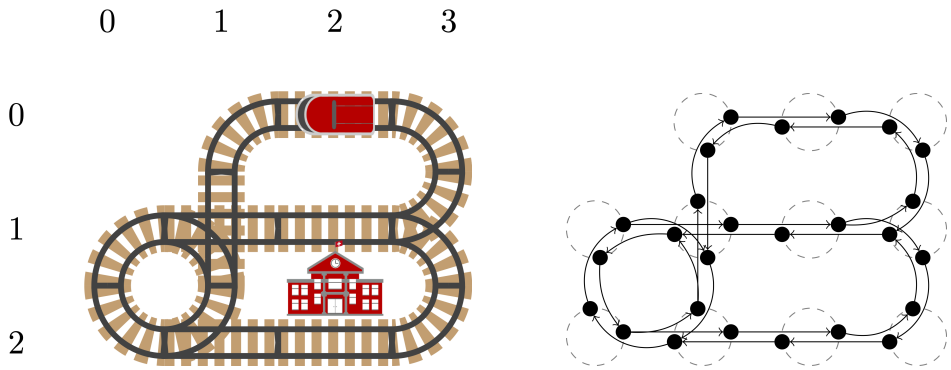
- **Starting position.** Each train is given a starting coordinate position $s$ on the grid. Additionally, it is given a starting direction $o$, which is used to determine the initial orientation of the train. An earliest departure time $\delta$ is also specified for each train, which is used to determine when the train can begin moving. However, the train does not have to depart at this time, and may wait at its starting position for any number of time steps before departing.
- **Goal position.** Each train is given a goal coordinate position $g$ on the grid. However, it is not given a goal direction, and may arrive at its goal position from any direction. A latest arrival time $\alpha$ is specified for each train, which is used to determine a deadline for when it should arrive at its goal position. In practice, this arrival time is not enforced as a hard constraint, but rather as a soft constraint that is used to evaluate the quality of a solution.
- **Speed.** Each train is given an inverse speed $v$, which describes how many time steps it requires to traverse a single cell. This means that a speed of $v = 1$ is the fastest possible speed, and corresponds to a train that can traverse a cell in a single time step. The next-slowest speed of $v = 2$ corresponds to a train that requires two time steps to traverse a cell.
- **Malfunctions.** During the course of a simulation, trains may experience malfunctions that cause them to stop moving for a certain number of time steps. These malfunctions are assigned randomly by the simulator, and are not known in advance by the participants. When a malfunction occurs, the duration of the malfunction is given, which specifies the number of time steps that the train will be unable to move. When an environment is initially created, users can specify the probability of a malfunction occurring, as well as the minimum and maximum duration of a malfunction.

## 5.4 Actions

- **Forward.** The action forward $f$ represents one of several situations:
  - a train, regardless of track type, has no directional decision and moves by following the topology of the track.
  - a train has a directional decision and moves forward.
  - a train spawns into the environment and appears in its starting cell.

  This action results in a new position but not always a new orientation.
- **Left.** The action left $l$ represents a train that has a directional decision and moves to the left, resulting in a new position and orientation.
- **Right.** The action right $r$ represents a train that has a directional decision and moves to the right, resulting in a new position and orientation.
- **Wait.** The action wait $w$ represents a train that retains its current position and orientation.

## 5.5 Graph translations




(a) Grid environment (b) Corresponding graph

Figure 5.2: Translation of a grid environment into its corresponding graph.

There are numerous ways to represent the grid as a graph, but we highlight one method here. Given a grid $C$, we define graph $\mathcal{G}(C) = (V, E)$, where:

1. $V = \mathcal{T}(C)$, where $\mathcal{T}(C)$ represents the set of transit pairs in $C$
2. $E = \{((c, d), (c', d')) \in V \times V \mid c \overset{d}{\hookrightarrow} c', (d, d') \in H_{t(c')}\}$

This kind of directed graph connects all transit pairs of a grid. A visual demonstration of the translation is shown in Figure 5.2. Connecting them in such a way allows us to account for the connectivity of different track types.

# Chapter 6

# Flaspland toolkit

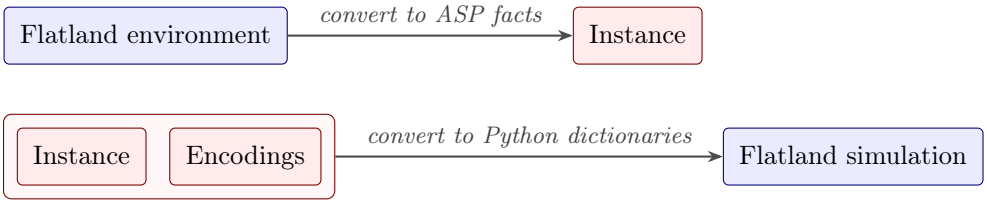



Figure 6.1: Overview of the Flaspland toolkit. Python components are shown in blue and clingo components in red.

Likewise, a description of the Flaspland toolkit is provided in our published work [7]. Since Flatland was developed using the Python programming language, a toolkit was developed to facilitate a translation between Python and clingo. The toolkit, whose overview is shown in Figure 6.1, serves two primary functions:

1. it generates Flatland environments based on user-defined parameters, which are then converted into a standard ASP format that clingo encodings can interpret.

2. it translates ASP models into Python objects that Flatland can interpret and runs the Flatland simulation to visualize the results of the encodings.

To this end, the toolkit seeks to enable users to focus on the development of ASP encodings for railway routing and scheduling problems, rather than on the details of the Flatland environment or the translation between Python and clingo. It additionally provides access to a set of benchmark environments derived from the actual Flatland challenge. These benchmark environments range in complexity from simple to challenging, by varying, among other things, the size of the environment, the number of trains, and the number of cities.

The key to success of the toolkit is adherence to a common fact format. When environments are generated, they are converted into a set of facts that describe the underlying structure of the environment. These facts are then used by the encodings to model the problem, and since they are represented in a consistent manner, encodings can be expected to work on previously-unseen environments. The following is the representation of an environment:

```
global(15).
start(0,(0,2),0,w).
end(0,(2,2),12).
```

```
speed(0,1).

cell((0,0),    0).  cell((0,1),16386).  cell((0,2),1025).  cell((0,3),4608).
cell((1,0),16386).  cell((1,1),38433).  cell((1,2),1025).  cell((1,3),6672).
cell((2,0),   72).  cell((2,1), 3089).  cell((2,2),1025).  cell((2,3),2064).
```

Likewise, the models generated by an encoding should adhere to a common format so that the translation back to the Flatland simulation can also be interpreted correctly. This enables a variety of approaches to be implemented, so long as the output adheres to the specified format: `action(train(ID),Action,Time)`. There are four possible actions: `move_forward`, `move_left`, `move_right`, and `wait`. The following is an example of what the expected output could look like:

```
action(train(0),wait,0).
action(train(0),wait,1).
action(train(0),wait,2).
action(train(0),move_forward,3).
action(train(0),move_forward,4).
action(train(0),move_forward,5).
action(train(0),move_forward,6).

action(train(1),wait,0).
action(train(1),move_forward,1).
action(train(1),move_forward,2).
action(train(1),move_right,3).
action(train(1),move_forward,4).
action(train(1),move_left,5).
action(train(1),move_forward,6).
```

To find out more about how to install and use the toolkit, please visit our repository at https://github.com/potassco/flaspland-toolkit.

# Chapter 7

# Baseline encodings

Since the more interesting research questions surround unsolved aspects of railway routing and scheduling, we provide baseline encodings that can be used to model the underlying behavior of the railway networks and the movement of trains within Flatland. Below, we introduce a few types of encodings: auxiliary, graph translations, and pathfinding. The expected workflow is that users take advantage of the auxiliary encodings to model the environment, then are encouraged to use the graph translations to represent the environment as a graph, and finally develop their own pathfinding encodings to model the movement of trains within Flatland. The provided pathfinding encodings are intended to serve as an example, but users are encouraged to develop their own pathfinding encodings to explore different approaches or to expand upon the problems being solved.

## 7.1 Auxiliary encodings

To model the behavior of the railway networks and the movement of trains within Flatland, as described in Section 5.1 and Section 5.2, we provide a set of auxiliary encodings that can be used to represent the underlying structure of the environment.

### 7.1.1 directions.lp

In the first auxiliary encoding, we define the set of cardinal directions and their relationships to one another. Much like the formalization in Section 5.1, the encoding defines the set of directions, their inverses, and their offsets.

```
dir(n).          dir(s).          dir(w).          dir(e).
inverse(n,s).    inverse(s,n).    inverse(w,e).    inverse(e,w).
offset(n,-1,0).  offset(s,1,0).   offset(w,0,-1).  offset(e,0,1).
```

### 7.1.2 tracks.lp

Since there is a finite number of specific tracks in Flatland, we define them outright in a single program for use in other encodings. There are six possible bi-directional **connections**: `(n,s)`, `(w,e)`, `(n,w)`, `(n,e)`, `(s,w)`, `(s,e)`. Track `32800`, for example, which is a straight vertical track, has a connection `(n,s)`. Because of the final rule in the encoding, we only need to define one direction of each connection, as the other direction `(s,n)` is automatically generated. Tracks that have multiple connections are defined with multiple rules, one for each connection. A portion of the encoding is shown below:

```
% Track Type 1 - Straight
connection(32800,   (n,s)).
connection(1025,    (w,e)).

% Track Type 2 - Simple switch
connection(37408,   (n,s)).
connection(37408,   (s,w)).
connection(17411,   (w,e)).
connection(17411,   (e,s)).
connection(32872,   (n,s)).
connection(32872,   (n,e)).
connection(3089,    (w,e)).
connection(3089,    (w,n)).

% ...

connection(T,(D,D')) :- connection(T,(D',D)).
```

### 7.1.3 grid-functions.lp

In the final auxiliary encoding, we define a set of functions that can be used to model the relationship between cells in Flatland environments, by defining the adjacency and connectedness of cells, as described in Section 5.2. The key distinction between a transition and a connection is that a transition models the direction a train will be facing, whereas the connection models the specific boundary of a track. For example, a train that enters from the southern boundary of a track is facing north.

```
transition(T,(D,D')) :- connection(T,(D',D'')), inverse(D'',D).
transition(T,(D,D')) :- connection(T,(D'',D')), inverse(D'',D).

adjacent((X,Y),D,(X',Y')) :- cell((X,Y),_), cell((X',Y'),_),
    offset(D,DX,DY), X'=X+DX, Y'=Y+DY.

connected((X,Y),D,(X',Y')) :- cell((X,Y),T), cell((X',Y'),T'),
    adjacent((X,Y),D,(X',Y')),
    transition(T,(D',D)), transition(T',(D,D'')).
```

### 7.1.4 aux.lp

For the sake of convenience, all three are packaged into a single file that can be included in other encodings.

```
#include "./directions.lp".
#include "./tracks.lp".
#include "./grid-functions.lp".
```

## 7.2 Graph translations

To profit from the advantages of graph-based approaches to railway routing and scheduling described in Section 5.5, we provide a set of encodings that can be used to translate the underlying structure of the Flatland environment into a graph representation.

### 7.2.1 subnodes.lp

```
node((X,Y,D)) :- cell((X,Y),T), transition(T, (_,D)).

edge((X,Y,D), (X',Y',D')) :-
    node((X,Y,D)), node((X',Y',D')),
    connected((X,Y),D,(X',Y')),
    cell((X',Y'), T'), transition(T', (D,D')).
```

### 7.2.2 edge-functions.lp

```
edge(C,C') :- connected(C,_,C').
node(C;C') :- edge(C,C').

d(C,C',D) :- connected(C,D,C').
p(C,C',D) :- connected(C,D',C'), cell(C,T), transition(T,(D,D')).
```

### 7.2.3 hypergraph.lp

```
hyperedge(C,C',C'') :- connected(C,D,C'), connected(C',D',C''), cell(C',T),
    transition(T,(D,D')).

node(C;C';C'') :- hyperedge(C,C',C'').
```

## 7.3 Pathfinding encodings

We offer a set of encodings in the `flaspland-toolkit` that can be used to model the pathfinding of trains within Flatland. While the above encodings are recommended for use in other encodings, the pathfinding encodings are provided as a starting point. For example, encodings can be developed to focus on specific aspects of the problem, such as optimization, varying speeds of trains, or for general computational performance enhancements.

In general, we recommend maintaining a modular approach to the development of encodings, as with the encodings provided above. One such example could be:

1. `aux.lp` to represent the underlying structure of the environment
2. `subnodes.lp` to translate the environment into a graph representation
3. `pathfinding.lp` to manage the pathfinding of trains within the graph
4. `constraints.lp` to model the avoidance of collisions between trains

5. `optimization.lp` to optimize the paths for some factor, such as by minimizing the total time taken for all trains to reach their destinations

Such an approach allows for the development of encodings that can be used to explore different aspects of railway routing and scheduling problems, while also allowing for the reuse of encodings across different problems.

# Bibliography


[1] D. Abels, J. Jordi, M. Ostrowski, T. Schaub, A. Toletti, and P. Wanko. Train scheduling with hybrid ASP. *Theory and Practice of Logic Programming*, 21(3):317–347, 2021. doi:10.1017/S1471068420000046.

[2] J. Behrens, R. Kaminski, T. Schaub, T. Son, J. Švancara, and P. Wanko. Routing and scheduling in answer set programming applied to multi-agent path finding: Preliminary report. *CoRR*, abs/2403.12153, 2024. URL: https://arxiv.org/abs/2403.12153.

[3] U. Brännlund, P. O. Lindberg, A. Nou, and J.-E. Nilsson. Railway timetabling using Lagrangian relaxation. *Transportation science*, 32(4):358–369, 1998.

[4] X. Cai and C. J. Goh. A fast heuristic for the train scheduling problem. *Computers & Operations Research*, 21(5):499–510, 1994.

[5] C. Cebi, E. Atac, and O. K. Sahingoz. Job shop scheduling problem and solution algorithms: A review. In *2020 11th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, pages 1–7, 2020. doi:10.1109/ICCCNT49239.2020.9225581.

[6] J. Li, Z. Chen, Y. Zheng, S.-H. Chan, D. Harabor, P. J. Stuckey, H. Ma, and S. Koenig. Scalable rail planning and replanning: Winning the 2020 Flatland challenge. In *Proceedings of the international conference on automated planning and scheduling*, volume 31, pages 477–485, 2021.

[7] R. Murphy, A. Nemes, J. Romero, and T. Schaub. Flaspland: A testbed for routing and scheduling in ASP. In *International Conference on Logic Programming and Nonmonotonic Reasoning*, pages 349–364. Springer, 2026.

[8] S. Narayanaswami and N. Rangaraj. Scheduling and rescheduling of railway operations: A review and expository analysis. *Technology Operation Management*, 2(2):102–122, 2011.

[9] E.R. Petersen and R. Merchant. Scheduling of trains in a linear railway system. *INFOR: Information Systems and Operational Research*, 19(3):246–258, 1981.

[10] R. Stern, N. Sturtevant, A. Felner, S. Koenig, H. Ma, T. Walker, J. Li, D. Atzmon, L. Cohen, T. Kumar, R. Barták, and E. Boyarski. Multi-agent pathfinding: Definitions, variants, and benchmarks. *CoRR*, abs/1906.08291, 2019. URL: http://arxiv.org/abs/1906.08291.